

Fundamentos del diseño funcional adoptado

El sistema diseñado es el resultado de principios propios del paradigma funcional, basado en la transformación de datos evitando modificar variables globales y/o producir efectos secundarios sobre las estructuras existentes.

Los criterios del paradigma funcional en los cuales nos basamos son:

1. Principio de inmutabilidad:

No utilizamos variables globales para almacenar el estado del sistema. En lugar de ello, cada función realiza cálculos a partir de los argumentos recibidos y devuelve un resultado.

Ejemplo: la función timer devuelve el estado del semáforo en un tiempo dado, éste resultado devuelto no es un valor que durante la ejecución de la función se ha guardado en una variable global, sino que es el resultado de un cálculo realizado a partir de los argumentos recibidos.

2. Principio de composición:

Hemos diseñado funciones que reutilizan el resultado de otras. Esto favorece la modularidad, lectura, mantenimiento y reutilización de las funciones.

Ejemplo: la función ciclos-por-tiempo-json utiliza el resultado obtenido por duracion-ciclo-json, mientras que varias funciones adaptadas para JSON reutilizan los valores recuperados mediante cargar-configuracion y obtener-tiempo.

3. Uso de funciones puras:

La mayoría de las funciones diseñadas son puras, realizan cálculos sobre datos recibidos y no modifican información externa.

Ejemplo: transicion, timer, duracion-ciclo...

Justificación de la librería seleccionada en la FASE 2 (CL-JSON)

El criterio adoptado por el grupo para seleccionar la librería CL-JSON fue la intención de incorporar nuevos conocimientos relacionados con el manejo de archivos de configuración externos y el intercambio de datos en formatos ampliamente utilizados en aplicaciones modernas.

Consideramos que esta alternativa representaba un mayor desafío técnico ante la simple modificación del formato de representación de fechas.

Por otro lado, también nos interesó aportar mayor flexibilidad al sistema. Ya que, siendo este TPI una representación simplificada de una problemática real, consideramos que la solución debía permitir modificaciones en su configuración.

Bitácora de bugs de depuración

1. duracion-ciclo y recomendacion-ciclo

```
(defun duracion-ciclo(rojo amarillo verde)
  (print (+ rojo amarillo verde))
  (recomendacion-ciclo(+ rojo amarillo verde))
)

(defun recomendacion-ciclo(duracion)
  (cond
    ((< duracion 35)
     "Ciclo demasiado corto")
    ((> duracion 150)
     "Ciclo demasiado largo")
    (t
     "Ciclo en rango óptimo")
  )
)
```

En principio, las funciones duracion-ciclo y recomendacion-ciclo fueron diseñadas para que la primera llame a la segunda en el cuerpo de la función

```
[1]> (duracion-ciclo 90 6 120)

216
"Ciclo demasiado largo"
[2]> (recomendacion-ciclo 90)

"Ciclo en rango óptimo"
[3]> (recomendacion-ciclo (duracion-ciclo 90 6 120))

216
*** - <: "Ciclo demasiado largo" is not a real number
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead.
ABORT          :R2      Abort main loop
Break 1 [4]> |
```

Al hacer la llamada de duracion-ciclo con argumentos de tipo átomo (que representaban la regla de temporización) devolvía dos valores: la duración del ciclo en segundos y la evaluación de recomendación-ciclo, lo cual podía ser un poco confuso para el usuario.

Luego, si intentábamos utilizar el resultado de duración-ciclo para llamar a recomendación-ciclo (lo cual nos pareció que era el fin del requerimiento 4), ocurría el error que se puede ver arriba en la captura.

Por lo tanto, decidimos modificar la función duración-ciclo para que su única salida sea el resultado del cálculo de la duración total y así el usuario pueda decidir si utilizarla de forma aislada o como una composición de funciones junto a recomendación-ciclo

```
(defun duracion-ciclo(rojo amarillo verde)
  (+ rojo amarillo verde)
)

(defun recomendacion-ciclo(duracion)
  (cond
    ((< duracion 35)
     "Ciclo demasiado corto")
    ((> duracion 150)
     "Ciclo demasiado largo")
    (t
     "Ciclo en rango óptimo")
  ))
```

Verificacion:

```
[1]> (duracion-ciclo 90 6 120)

216
[2]> (recomendacion-ciclo (duracion-ciclo 90 6 120))

"Ciclo demasiado largo"
[3]> (recomendacion-ciclo 90)

"Ciclo en rango óptimo"
[4]> (recomendacion-ciclo 216)

"Ciclo demasiado largo"
```

2. distribucion-porcentual

```
(defun distribucion-porcentual(rojo amarillo verde)
  (list
    (list 'porcentaje-rojo (* (/ rojo 216.0) 100))
    (list 'porcentaje-amarillo (* (/ amarillo 216.0) 100))
    (list 'porcentaje-verde (* (/ verde 216.0) 100)))
  )
```

La función devolvía valores porcentuales que sumados en conjunto no representaban la totalidad porcentual de tiempo en que hubiera cualquier luz prendida. Es decir, a veces devolvía valores que parecían indicar que más del 50% del tiempo todas las luces estaban apagadas.

```
*REPL* [lisp] x
[1]> (distribucion-porcentual 90 6 120)
((PORCENTAJE-ROJO 41.666664) (PORCENTAJE-AMARILLO 2.777778) (PORCENTAJE-VERDE 55.555557))
[2]> (distribucion-porcentual 90 6 12)
((PORCENTAJE-ROJO 41.666664) (PORCENTAJE-AMARILLO 2.777778) (PORCENTAJE-VERDE 5.555556))
[3]> (distribucion-porcentual 2 3 5)
((PORCENTAJE-ROJO 0.9259259) (PORCENTAJE-AMARILLO 1.388889) (PORCENTAJE-VERDE 2.3148148))
[4]>
```

Por ejemplo, en Break [3]

- ➔ Rojo: 0.9%
- ➔ Amarillo: 1.3%
- ➔ Verde: 2.3%

Eso representa que del 100% de cualquier valor de tiempo dado, aproximadamente solo un 4.5% del mismo prendió alguna de las 3 luces.

Después de una breve observación pudimos detectar el error...

La función podía recibir como argumentos diferentes reglas de temporización, pero en el cuerpo solamente se calculaban los porcentajes correctos para los ciclos que duraran 216 segundos.

Nuestra decisión fue definir una variable local en la que se guardara el tiempo que duraría el ciclo y así sacar el porcentaje adecuado para la regla de temporización que el usuario quisiera utilizar.

```
(defun distribucion-porcentual(rojo amarillo verde)
  (let ((total (+ rojo amarillo verde)))
    (list
      (list 'porcentaje-rojo (* (/ rojo total) 100.0))
      (list 'porcentaje-amarillo (* (/ amarillo total) 100.0))
      (list 'porcentaje-verde (* (/ verde total) 100.0)))
    ))
```

Verificación:

```
[1]> (distribucion-porcentual 10 90 12)
((PORCENTAJE-ROJO 8.928572) (PORCENTAJE-AMARILLO 80.35714) (PORCENTAJE-VERDE 10.714286))
[2]> |
```

Informe del lenguaje asignado (Erlang):

Erlang es un lenguaje de programación de código abierto disponible en <http://www.erlang.org>.

Fue creado por el laboratorio de ciencias de la computación de Ericsson en la década de 1980 comenzando como una versión modificada de Prolog, que es de donde proviene su sintaxis, mientras que su operador de envío de mensajes proviene de CSP.

Es un sistema de tiempo flexible, ya que es difícil garantizar exactamente cuanto tardará en ejecutarse una acción, esto se debe a que los procesos de Erlang son muy ligeros a su vez de que cada uno se recolecta de forma independiente.

A su vez posee una alta tolerancia a los errores con una estrategia basada en módulos “supervisores” los cuales son los encargados de iniciar, detener y supervisar los procesos “trabajadores”, reiniciándolos al detectar algún error. Esta explicación da un vistazo ligero al sistema de supervisores, por lo tanto, recomendamos encarecidamente visitar https://www-erlang-org.translate.goog/doc/apps/stdlib/supervisor.html?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true para indagar profundamente en esta estrategia.

Rubro más utilizado

Actualmente Erlang domina principalmente en los sistemas y aplicaciones de telecomunicaciones así como en bases de datos que requieran modelos de tiempo real flexible, ya que al poseer una alta tolerancia a los errores debido a su modelo de procesos supervisados, la ligereza de sus procesos, la concurrencia y el paso de mensajes propios de Erlang y las bibliotecas OTP (Open telecom platform) que brindan soporte excelente para errores comunes en dichos sistemas.

Algunas de las empresas más reconocidas que utilizan el lenguaje:

- Facebook (sistema de chat)
- WhatsApp
- AdRoll
- EE

Preguntas adicionales sobre el lenguaje:

- 1. Erlang utiliza una sintaxis derivada de Prolog. Muestren cómo se delimitan las funciones y las expresiones (puntos, comas y puntos y comas) y cómo afectó esto a la lectura del código comparado con los paréntesis de Lisp.

- Las funciones poseen **dos** formas particulares de delimitarse, las cuales dependen de si son parte de una **cláusula** (es decir, la función **continua** luego de chequen una condición) o, por el contrario, finalizan la función completamente, siendo que en el 1er caso se utiliza “;” mientras que el “.” Se reserva para cerrar la función. Por otro lado, las expresiones se separan a través de una “,”.

Si bien la sintaxis es considerablemente diferente a Lisp, no fue un reto acostumbrarse a esta, ya que posee características similares a lenguajes que hemos visto previamente, como C (delimitación mediante signos de puntuación), sin embargo, nos resultó más sencillo al momento diferenciar las separaciones de expresiones comparado al sistema de paréntesis de Lisp. Es importante mencionar que la capacidad de incluir “cláusulas” añadió una nueva forma de crear funciones, provocando que debamos adaptarnos a esta nueva lógica.

- 2. En Erlang las variables son de *asignación única*. ¿Cómo simulan un ciclo continuo de cambios en el semáforo si no pueden reasignar valores a las variables?
- Dicho ciclo puede ser simulado a través del uso de la función f(), la cual desvincula las variables para una nueva asignación, o a través del sistema de cláusulas, simulando una recursividad con llamadas sucesiva, en las que el estado anterior se pasa como parámetro

Ejemplo:

```

“semáforo(tiempo) when tiempo < 90 →
‘En-rojo’
semáforo (tiempo + 1);
semáforo(tiempo) when tiempo > 90, tiempo < 96 →
‘En-amarillo’
semáforo (tiempo + 1);
semáforo(tiempo) when tiempo > 96, tiempo < 216 →
‘En-verde’
semáforo(tiempo + 1);

semáforo(tiempo) →
semáforo(tiempo * 0).

```

Conclusion:

El estudio de Erlang nos permitió acercarnos todavía más al enfoque recursivo de la programación funcional y experimentarlo en un lenguaje diferente al utilizado durante el desarrollo principal del proyecto. Su sintaxis nos resultó interesante y, aunque la escritura de funciones mediante cláusulas puede resultar más extensa que en otros lenguajes, también nos pareció bastante intuitiva una vez comprendida la lógica recursiva sobre la que se apoya.

Para concluir, la experiencia de aprender una nueva forma de programar y, en general, la de trabajar en este proyecto, resultó enriquecedora. Nos permitió aplicar conceptos teóricos en un caso práctico, enfrentarnos a problemas reales de implementación y ampliar nuestros conocimientos sobre programación funcional, aprendizajes que consideramos de gran valor para nuestra formación y desarrollo profesional futuro.

Integrantes:

- Gabriel Esquivel
- Gabriel Fava
- Fabio Javier Fernández
- William Francisco Cantero

Fuentes:

(Git y GitHub)

<https://git-scm.com/docs>

<https://docs.github.com/es/get-started>

(Quicklisp y CL-JSON)

<https://www.quicklisp.org/beta/>

<http://github.com/stablecross/cl-json-helper/blob/master/README.md>

(LISP)

<https://lisp-docs.github.io/cl-language-reference/chap-21/cb-b-stream-concepts>

(Erlang)

<https://www.erlang.org/faq/introduction.html>

<https://www.erlang.org/faq/academic.html>

<https://www.erlang.org/doc/apps/stdlib/supervisor.html>